

A Failure to Learn By Software Developers: Inhibiting the Adoption of an Agile Software Development Methodology

John McAvoy and Tom Butler

University College Cork, Ireland

j.mcavoy@ucc.ie

tbutler@afis.ucc.ie

ABSTRACT

The adoption of an Agile software development methodology can be impacted by many stakeholders. In the parlance of Chris Argyris, the adoption of Agile is an exercise in learning - behavioral change created by Agile's values. This paper presents a case study investigating the failures associated with the introduction of elements of a new software development methodology by the software developers within a software development team. Although Argyris' work is often portrayed at the organizational level, this case study examines individual learning. The failure to adopt aspects of the new methodology is seen as a failure to learn by the individual software developers within the team. This paper posits that learning is more than a pedagogical view of learning a new skill; learning also involves changes in individual behaviors, attitudes and opinions. As methodology adoption involves changes to the software developers' activities and values, this study investigates the introduction of an Agile Method as a learning experience. This case study concentrates on one particular group of stakeholders – the software developers – and examines the affect their traits can have on the learning activity of adopting an Agile methodology in a project. Extant research focuses on the traits of software developers but does not incorporate influences from the values inherent in Agile. The objective of this case study is to examine the traits of individual programmers to determine the impact they can have on the adoption of an Agile methodology by examining, in depth, the introduction of Agile by the software developers in a team. This leads to the research question which asks if and how the traits of software developers can negatively impact on the behavioral changes required for the learning inherent in an Agile adoption?

Keywords: Learning, Behaviors, Agile Adoption, Software developers, Cognitive dissonance

INTRODUCTION

Previous research illustrates how learning occurs and fails to occur: to be effective, learning involves a change in behaviors and beliefs (Argyris, 1976). The new deployment of a software development methodology is an exercise in change – a change in actions that lead to new behaviors (Larsen, 2001). Such change is, in effect, an exercise in learning. Previous research examined learning in the context of a development group (cf. McAvoy & Butler, 2007). While the original research guides much of the discussion on learning, this study examines the introduction of an Agile method (usually referred to simply as Agile) in the context of individual

software developers, and determines why their individual learning experience was at times ineffectual.

The remainder of this paper is organized as follows. First, the Agile method and philosophy are described. Then, theories of how learning occurs in individuals are presented. Next, a relationship between the values of Agile and learning behaviors is posited; it is also argued that the values inherent in Agile should promote learning. Subsequently, the traits of software developers are presented as possible moderators of the behavior changes required for an Agile adoption and the learning inherent in the adoption. This leads to the research questions, which ask if and how the traits of software developers can affect the behavioral changes essential for the learning required for effective Agile adoption? The case study of the adoption of an Agile method is then presented, the findings of which illustrate how the traits of individual software developers, paradoxically so highly regarded by Agile methods, inhibited learning in the project. The final section offers conclusions and recommendations.

AGILE VALUES AND METHODOLOGIES

The Agile philosophy is encapsulated in the Agile manifesto. The agile manifesto, available at <http://agilemanifesto.org>, is a collection of values, or espoused values (Wendorff, 2002), that underlie all agile methodologies.

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Agile places a much greater emphasis than other methodologies on software developers by concentrating on the humanistic aspects of software development (Stephens & Rosenberg, 2003). People issues are at the core of the Agile methodologies and this is evident in the levels of empowerment that software developers possess (Boehm & Turner, 2004). Developers are empowered because, rather than rigid plans and documentation controlling the development process, the Agile methods move the ownership and control of the project back to the software developers. The move to empowerment of skilled software developers acts in tandem with new attitudes from the project stakeholders towards change; change is a positive aspect of a project as opposed to being perceived as a problem to overcome (Riehle, 2001). Along with a new set of values, the Agile methods introduce new processes or methods to software development. These processes fit with the values of Agile as described in the manifesto above. An example Agile process, and the most written about, is pair programming. Pair programming is described in Hazzan & Tomayko (2003 p.55), Maurer & Martel (2002), Natsu *et al.* (2003), and Turk *et al.* (2005) as a practice where “any piece of code should be written by two software developers, each of whom has a different role.” While one software developer is typing, the other has a strategic outlook, determining what the best approach to the problem should be.

AGILE AS AN EXPERIENCE IN LEARNING

Researchers and authors have described the introduction of Agile as a culture change (cf. Bossavit, 2002; Coram & Bohner, 2005; Drobka, Noftz, & Raghu, 2004; Simons & Nadkarny,

2002). This paper notes this observation and therefore concentrates on the change brought about by the introduction of Agile through the lens of the learning experiences (behavioral changes) of the software developers.

Argyris (1976) describes learning as the detection and correction of errors. In a later work, Argyris (1995) adds that learning also occurs when there is a match between people's intentions and the consequences of their actions. Individuals and teams introduce Agile with the aim of creating better software faster; this is achieved by following the values of the Agile manifesto. Often, the decision to introduce Agile is based on the need to correct problems with an existing, more rigid, methodology (Schuh, 2001; Spayd, 2003)

Learning and Behaviors

Argyris and Schon (1978) illustrate the relationship between intentions and consequent behavior by connecting learning and behavior through espoused theories (intentions) and theories in use (consequences or realities). Espoused theories are the intentions that individuals describe, and conceive themselves as having – for example the intention to introduce Agile to a project. Theories in use guide how individuals actually behave – what intentions actually guide social actors, as opposed to what they profess guides them. Espoused theories comprise beliefs and attitudes which influence intentions. In addition to learning requiring that intentions correspond with consequences, learning also occurs where there is a match between the beliefs individuals profess, to themselves and others, and the beliefs that can be construed from their actions (Easterby-Smith & Lyles, 2003) – for example a manager may profess a belief in delegation of decision making (their espoused theory), yet their behavior is one where only trivial decisions are delegated. This implies that the manager actually believes that delegation of decision making is not for important decisions (their theory in use).

Single and Double Loop Learning

Argyris (1994; 1995; 1997; 2002) argues that there are two forms of learning—single and double loop. Argyris uses the often-cited example of a thermostat to demonstrate single loop learning. In this scenario, temperature is measured against a predefined value and the thermostat turns on or off the heat source to maintain the temperature at this value. In single loop learning, the norms and beliefs that chose the temperature are unchanged—for example, the reason why the temperature is set to 28 degrees as opposed to 32 degrees. Double loop learning occurs when strategies are examined, along with the assumptions behind those strategies. Using the original analogy of a thermostat to demonstrate single loop learning, the analogy of the thermostat is further developed by stating that if the thermostat could question why it was set to a temperature, or to examine why it worked a certain way, then the thermostat would be engaged in double loop learning (Argyris, 1997, 2002; Blackman, Connelly, & Henderson, 2004; Denhardt, Denhardt, & Aristigeuta, 2002). Double loop learning not only questions the facts, it questions the logic behind the facts: thus, norms and values are questioned leading to the emergence of new theories in use (Argyris, 1995; Argyris & Schon, 1978; Blackman *et al.*, 2004; Bokeno, 2003; Easterby-Smith & Lyles, 2003).

The use of an Agile methodology appears to promote opportunities for double loop learning as opposed to single loop learning – this is, in theory, a positive benefit of adopting Agile. It should

be noted that this can only be implied, as there is no extant research investigating a link between Agile and double loop learning. Despite the lack of extant research, existing literature on single and double loop learning could be interpreted as implying that Agile encourages and creates double-loop learning (see table 2). It should be noted, though, that this study argues that the implied link between Agile and double loop learning can actually be negative; however, the review of existing literature above would strongly suggest a positive link.

As described above, Argyris & Schon (1978) illustrate the relationship between intentions and consequences by connecting learning and behavior through theories in use (consequences) and espoused theories (intentions). Human behavior can be categorized as following a particular theory in use, typically Model I behavior. Model I behavior is operationalized by Argyris as the need to win at any cost, the suppression of negative feelings, an emphasis of rationality, stability, and the avoidance of risk. This type of behavior leads to the desire to protect the individual from criticism, which leads to individuals being overly defensive: this has the consequence of choice reduction and the elimination of mechanisms for valid feedback. As a result, individuals exhibiting Model I behavior have little genuine feedback on, or testing of new ideas. Argyris (1976; 1994; 1995; 1997; 2002) describes this behavioral model as being underpinned by single loop learning, as by censoring what is said and heard, double-loop learning is prevented. Model II behavior moves away from the repressive censoring behaviors of Model I. The governing variables of Model II behavior are valid information, free and informed choice, and commitment to decision and actions. This model is based on double loop learning and is a move away from the concept of winning to one of open debate. The comparison between Model I and Model II can be seen in Table 1.

Model I / Single Loop Learning	Model II / Double Loop Learning
Suppression of negative feeling. Limited feedback. Censorship.	Valid information
Importance of rationality, stability, and the avoidance of risk. Rationality is used to justify one's own views and discount those of others.	Free and informed choice / Open debate
Winning (not losing or being seen as incompetent)	Commitment to decisions and actions

Table 1: Comparison of Argyris' Model I and Model II behavior

Salaway (1987) warns that Model 1 behavior is not automatically bad behavior, and Model II behavior is not necessarily good. Model I behavior is efficient in dealing with routine problems – unlike typical Agile projects – while Model II behavior is appropriate where there is a need for considerable cooperation and co-dependence – as with Agile projects. Agile, therefore, should strive for and require Model II behaviors. The governing variables for Model II behavior, and the

double loop learning it is based on, appear to be matched by the values of Agile, as seen in Table 2: this proposes a relationship between the two. It should be noted that, although Argyris uses Model II and double-loop interchangeably, Bokeno (2003) is more specific by stating that, ideally, Model I behavior leads to single-loop learning, while Model II behavior leads to double-loop learning.

Model II Governing Variables Double Loop Learning	Agile values
Valid information	“The agile manager fosters an environment where members of a team are able to make decisions and base them on the best information available” [Schuh, 2004, p. 164]. Agile meetings are held to ensure sharing of information useful to the team; superfluous information and discussions are banned (Astels et al., 2002; Boehm & Turner, 2004; Poole, 2004)
Free and informed choice / Open debate	Agile projects should have rules governing behavior including: Everyone should have an equal voice; Everyone’s contribution is valuable; Attack issues not people; Keep privacy within the team; Respect each other and your differences; Everyone participates (Highsmith, 2004).
Commitment to decisions and actions	As everyone is involved though participative decision making, the software developers will implement the decisions that are made (Highsmith, 2004) A decision is implemented if it is accepted by those that make it (Von Bergen & Kirk, 1978; Vroom, 1974).

Table 2: Double loop learning and Agile

Argyris highlights that Model I behavior is the norm in most situations. Double loop learning is thus a rare occurrence in individuals and teams as single-loop learning appears to be the norm. The failure to achieve Model II behavior is, therefore, a failure to achieve double-loop learning. It would appear from Table 2 that the use of an Agile methodology seems to favor and enhance opportunities for double loop learning by permitting software developers to transcend the limitations of Model I behavior. In an ideal situation, therefore, the use of Agile should demonstrate double loop learning seen through Model II behavior.

Although Agile should promote, and appears to assist, Model II behavior, not all Agile adoptions are successful. Success here can mean that the project delivers software on time and on budget, meeting requirements. It can also mean that the Agile methods are properly applied and used. If the Agile principles are not applied, this can be considered as a learning failure. Coram & Bohner (2005) and Lindstrom & Jeffries (2004) describe patterns of success and failure in Agile projects. Rumpe & Schroeder (2002) show how the majority of XP (an Agile methodology) adopters they surveyed found their projects to be successful, yet examples of failures in organizations adopting Agile are provided in Bossavit (2002), Hodgetts (2004), Schwaber (2002), and Grossman *et al.* (2004). In some cases, therefore, there can be ineffective learning in Agile projects.

If, as argued herein, the attributes of Agile match those of Model II behaviors, then a failure to adopt Agile by the software developers is essentially a failure to achieve double-loop learning. Back & Seaker (2004) argue that an individual's personality influences their ability to achieve double loop learning, so this study concentrates on the personalities and traits of software developers to determine their impact on learning. At the core of Agile software development are the software developers, so any impact on Agile adoption (the behavioral changes inherent in learning), in this context, will be a major one. In the next section we examine factors which may impact on the introduction of an Agile methodology, factors which may affect learning as defined by behavioral change.

POTENTIAL CONCERNS WITH THE PROMINENT ROLE OF SOFTWARE DEVELOPERS IN AN AGILE PROJECT

There is a stereotypical view of software developers which is often summed up by the word 'geek'. Research, though, has demonstrated that this stereotype is often based in reality (Licker, 1985). Couger *et al.* (1979) found that software developers have a very low Social Need Strength (SNS) – the need to interact with others – compared to other professional groups (Capretz, 2002; Couger & McIntyre, 1988; Licker, 1985; Sawyer & Guinan, 1998); as such, they tend to be 'loners' rather than 'team players'. Researchers and other commentators have labeled software developers as anti-social (Yourdon, 1993), lacking in people skills (Wieggers, 2001), introverted (Badawy, 1997), and socially repressed (Olson & Stimmel, 2002). Such software developer characteristics would appear to present a problem for the adoption of an Agile methodology, especially as software developers working closely with their colleagues and customers is stressed as a critical factor in the success of an Agile project (Highsmith, 2004). Model II behavior, based on double loop learning, is grounded in team goals, so an inability of software developers to work in teams could be problematic for their individual learning - the inherent behavior changes required for an Agile approach.

How software developers are motivated can affect the success of the learning associated with the introduction an Agile methodology. Somewhat paradoxically, software developers are motivated by recognition from their peers; this is despite their low SNS as described above. Software developers place a lot of value on a good reputation among their peers (Raymond, 1999), and try to impress their peers with their programming skills (Sawyer & Guinan, 1998). Recognition from peers, though, would not mitigate a further problem with software developers and their dealings with others – customers specifically. Technical employees tend to be blind to what customers need (Workman, 1997). They can get into arguments with customers over technical approaches,

their attitude being described as arrogant. Software developers regard their views as being superior to those of the users (Cooper, 2004), and they develop systems that do what the software developers themselves want. As customers are considered to be at the core of an Agile project (Turk et al., 2005), this has the potential to present a major problem for Agile projects. Developer interaction with the customer is regarded as key to success.

Unfortunately, for those wishing to adopt an Agile methodology, the traits exhibited by these stereotypes do not appear to be the only ones to have a potentially detrimental impact on the learning/behavioral changes required for an Agile adoption. How an Agile project is managed will also affect the success of an adoption of a new methodology, as it heavily influences the software developers. Agile project management has been defined as the empowerment of the software developers to rapidly deliver value to the customer (Augustine, 2005). Traditional project management would attempt to control the negative impacts of the developers' traits by imposing a standardized structure which relies less on the developers' skills and abilities. By empowering the developers, which is a core element of Agile project management, the developers, and their traits, have a greater impact in the project. A further concern with this new approach of empowering the software developers derives from the fact that project management has traditionally followed an engineering or scientific approach (Boody & Buchanan, 1992; Humphrey, 1996), and assumes that work is done in a systematic way by software developers, who themselves follow an engineering approach (Capretz, 2002). Nevertheless, questions have been raised as to whether software developers really follow an engineering approach. Software developers have been described as artists, or at least believing themselves to be artists (Jones & McLean, 1970).

Perceptions of software developers as artists is seen as problematic, and even described as anarchy (Yourdon, 1993): thus software developers have been described as unmanageable because of their traits (Licker, 1983). In a more recent work, Howard (2001) adds that it can be difficult to manage software developers who resent the imposition of rules or processes, especially creative and knowledgeable people (Acuna & Juisto, 2004). The project management of a collection of individuals who perceive themselves as artists is therefore a potential problem for the introduction of an Agile project. Project management in an Agile software development initiative moves away from an engineering approach to one of reliance on the software developers. Project management has traditionally operated from an industrial engineering approach (Carmel & Sawyer, 1998), where all tasks are broken down and performed systematically – akin to a bureaucratic approach epitomized by single loop learning. Making the transition from this rigorous approach to one of trusting in the abilities of the software developers that constitute an 'artists' colony' raises several concerns.

A potential problem with an Agile software developers as a 'colony of artists' has been highlighted by Weinberg (1971), who provides one differentiation between poet/artist and software developer – that is in the evaluation of work. A critique of an object of art or a piece of literature can be good or bad, depending on the critic's taste. A program, on the other hand, can be judged by running it – it will work or not (Weinberg, 1971) or, more specifically, it will meet the requirements of the user. Software developers, with their artistic temperaments, are notoriously bad at evaluations. A prime example of this is in others evaluating their progress, or

lack of viz. software developers (as a class) “are often a dour and apologetic lot who are constantly explaining why things can’t be done on time and up to specification” (Coffee, 2001, p.187). This appears to match the description of Model I behavior.

There are, nonetheless, additional arguments that this artistic perspective can have a detrimental effect on development practice, especially in the introduction of an Agile method and the inherent learning within the adoption. Software developers that debug their own programs can indulge in blame shifting, denial, excuses, and even ignoring the problem (Hunt & Thomas, 2000). This is characterized as ego protection, which again matches descriptions of Model I behavior. Thus, a further concern for those introducing an Agile methodology is raised, as the role of testing lies with the software developer (ultimately it lies with the customer but is initially a concern of the software developer). With agile methods, software developers are required to continually test their own code; The separate role of tester does not exist in Extreme Programming (one of the major Agile methods).

Weinberg took the existing theory of cognitive dissonance and used it to explain why software developers cannot test their own work (contrary to those who advocate Agile methods, although written before their advent). The original theory of cognitive dissonance was described by Festinger, in the 1950’s, and has been used more recently in connection with software developers (cf. Hunt & Thomas, 2002; Koutsoukos, 2006). Cognitive dissonance occurs when individuals experience dissonance and avoid it. The dissonance (or conflict in a person’s mind) is where a person’s self image, or understanding, is challenged and is avoided by adjusting beliefs. In a review of research into cognitive dissonance, it was noted by researchers that research has shifted from a need to maintain logical consistency in beliefs, to recognition of the importance of maintaining self-esteem (Greenwald & Ronis, 1978). In the 1970’s, Weinberg applied the cognitive dissonance theory in an examination of software developers. Weinberg describes the situation where software developers test their own code. Cognitive dissonance implies that the software developers will not see what they don’t want to see – in this case, bugs in their code (Weinberg, 1971).

RESEARCH MODEL AND QUESTIONS

Although there is no extant research examining how the traits of software developers affect the learning/behavioral changes required in the introduction of an Agile methodology, it appears that literature from other domains suggests that there may be concerns. These are potentially the factors which cause Model I as opposed to Model II behavior. This research therefore examines if the traits of software developers can impact upon the adoption of an Agile methodology, as viewed through the eyes of the behavior change inherent in learning.

Learning is often viewed as a pedagogical issue: learning a new skill or teaching students in a classroom. Learning in this research is based on Argyris’ concept of the behavioral change required of the individual. It is proposed that the traits of software developers can inhibit double-loop learning, so the research questions to be answered are

- 1. Can the traits of software developers impede the double-loop learning and Model II behavior required in the adoption of Agile? If so:**

2. Which traits create adoption problems?
3. How do these traits impede adoption?
4. What can be done to mitigate the negative impacts of these traits?

The model shown in Figure 1 describes how learning is achieved by individuals changing from Model I behavior to Model II behavior. A successful Agile adoption requires successful learning, so a successful adoption will have inherent changes to Model II behavior; it is proposed here that the traits of software developers can impede this learning. The model shows how the Agile philosophy can play a positive role in the move to Model II behavior (as can peer motivation), but other developer traits can negatively impact on the success of the adoption, and the learning required. Learning itself cannot be seen, but by observing behavioral change from Model I to Model II, it can be implied. Similarly, if the traits of developers are impeding learning, this will be observed by a prominence of Model I behavior over the Model II behavior required for a successful Agile adoption.

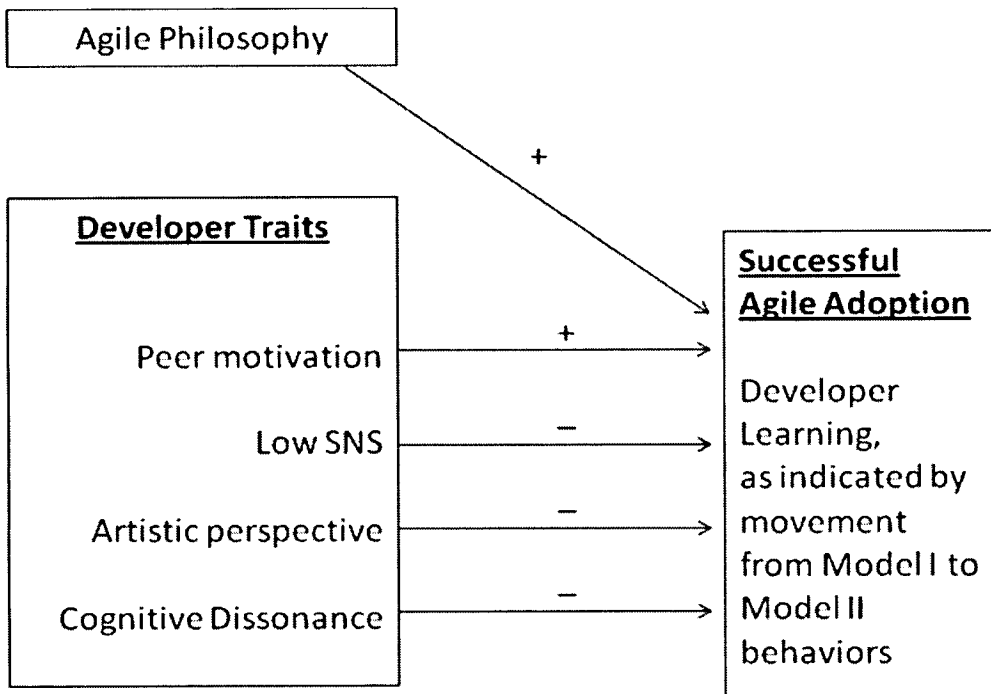


Figure 1: Research Model: Software developers' traits preventing learning

RESEARCH APPROACH

This paper's basic premise is that the successful adoption and use of Agile methods is related to the ability of software developers to adopt Model II behaviors. The objective of this study was,

therefore, to examine how software developers learned (in the context of behavioral changes as opposed to pedagogical learning), or did not learn, in the introduction of an Agile method. Factors which aided software developers' learning and, also, those which constrained learning were to be identified and explained. Klein & Myers (1999, p.75) maintain that in the interpretivist mode of research "theory is used ... more as a 'sensitizing device' to view the world in a certain way"; hence the inclusion of previous sections on learning theory.

A qualitative, interpretive, case-based research strategy was therefore adopted (cf. Walsham, 1995), using a single case for analysis (cf. Yin, 2003)—a software development project team working in the software division of a large US multinational telecoms manufacturer. Stake (1995) contends that case studies can be intrinsic, instrumental, or collective in focus. An instrumental case study is one where the case forms a backdrop or context and is of secondary importance. Such a study is undertaken to build or extend a theory or to explore a particular phenomenon as it is manifested in the case—that is the purpose of the present study. The software team was studied over a nine month period, with one of the researchers playing an active role in the team in facilitating the deployment of the Agile methodology. In this capacity he was both a 'fly-on-the-wall' and an active participant at key stages (cf. Yin, 2003). Social actors, such as senior project managers, section managers, and members of other software development teams were also included in the case study to gather background information on the team studied.

As indicated, problems with learning and behaviors are often latent or hidden, even from those involved in the process. Individuals in groups, such as software development teams, are usually unable to determine why these problems occur, for a variety of social and psychological reasons mentioned above. This situation calls for the application of research approaches that are sensitive to such issues (Jorgensen, 1989). Thus, to investigate the impact on learning, by software developers in an Agile adoption, necessitates a longitudinal case study that permits the team to be examined in context and over time (as per Yin, 2003). Furthermore, the investigation of the software developers' traits that inhibit learning involves the examination of phenomena that would be normally hidden from, or not observable by, outsiders; indeed, individuals themselves may not be aware of the existence of such influences (Goleman, 1996). In addition, as software development (and Agile software development in particular) occurs in a social setting, it brings with it the complexities inherent in social settings. When dealing with such complexity, case based research is recommended (Gordon, 2008); examples of the use of case studies to examine a social setting can be seen in Furumo & Melcher (2006), who found the impact of social structure on an ERP implementation, and Joshi (2005) who examined the impact of the user on the adoption of a system.

Further, Argyris (1976) illustrates how individuals are unable to discern the difference between what they believe in and what they actually do ('espoused theories' versus 'theories in use') and it is clear from Argyris that phenomena such as the inability for reflexivity are usually hidden from those affected by the phenomenon. Surveys, questionnaires, and other such approaches are therefore unsuitable research methods as a respondent cannot, or may not wish to, identify the differences between their 'espoused theories' and 'theories in use'. Thus, to overcome these difficulties, a qualitative research approach utilizing participant observation was adopted for the case study (cf. Yin, 2003).

Case studies have been used to examine software development, and it is argued that it is the most appropriate method. Kitchenham *et al.* (1995) propose case studies as the optimum research approach when investigating the introduction of a new tool or method in software development. In their investigation into software processes, Curtis *et al.* (1988) noted that many of their research unit (The Microelectronics and Computer Technology Corporation) were proponents of quantitative experimental research. Curtis *et al.*, though, favored research in the field to enable timely insight into problems experienced by companies. In support of their choice, they argue that quantitative studies concentrate on the processes and mechanisms involved, yet software is developed by humans and the problem is a behavioral one. In addition, it is argued that there is a need for research on systems development in real organizational settings (Fitzgerald, 1997). Specific to Agile software development, Layman *et al.* (2006) argue that most reports, from practitioners and researchers, on Agile success has been anecdotal, a point with which Maurer & Martel (2002), Layman *et al.* (2004), Turk *et al.* (2005), and Kahkonen & Abrahamsson (2003) concur, and argue for a case based approach to provide sound empirical evidence. A number of case studies have employed participant observation to conduct research on software development projects. For example, a qualitative approach involving a participant observation case study was used by researchers to investigate the characteristics of an Agile software development team and provided rich insights that could not be obtained by other research methods (Robinson & Sharp, 2005).

In the present study, one of the researchers was previously employed by the organization studied. Thus, he had ready access to key informants and a range of other social actors in the case. This conferred upon him the advantage of being a 'cultural insider' (Bødker & Pedersen, 1991). Molla & Bhalla (2006) describe a similar case study approach, using participant observation. One author was a cultural insider, having worked in the company, while the second author examined the data for potential bias. Molla and Bhalla's research differs from this research in that they describe their participant observation as being retrospective (being based on one author's previous experience), while the research in this case study occurred in 'real time'; the team was researched over the nine month period of the project in which Agile software development was introduced.

Extensive case and field notes were taken on all formal interviews, meetings, informal conversations, and observations while on-site at the research location (Ezey, 2003). A wealth of documentary evidence was also gathered. All project documentation was made available to the researcher, from process documents to performance statistics. While observation was the primary data source, supplemental and confirmatory information was acquired through documents and interviews. Each member of the team was interviewed, often on several occasions, at various stages of the project. While the majority of the interviews were conducted with individuals, the entire group was also interviewed as a unit, to determine if there were differences between public statements and individual realities – espoused views in the group setting and theories in use of the individuals. The goal was not to examine group behaviors but to determine the impact of the team on individual behavior (and the learning inherent in behavior change). Observation took place over the nine month period, the duration of time in the field was predominantly a full working day every week (on alternate days); this led to a total observation time of approximately 300 hours. On occasion, when additional information and clarification was required, individual team

members would meet with the researcher outside of the work setting for short interviews. Observation was not restricted to the work place, as the researcher was invited to several social events outside of the work place. These events took place in restaurants, bars, and even the homes of the software developers. This proved to be the most difficult for observation as it was not possible to take notes during the social events – at a social event people would not want someone recording their actions – so notes were written after the event. In the workplace setting, the individual software developers agreed not to request to see the field notes from observations as they were being recorded. Once the notes had been written up, the software developers could see them, although only once was this requested and the software developer had no issue with the observations in the field notes. During formal interviews, the software developers were shown the transcription of their interviews as a matter of course.

Following Yin (2003), an analytic generalization approach was employed with saw the data interpreted in light of the theoretical concepts described above. Thus the data analysis was informed by the conceptual elements (e.g. ‘espoused theories’ and ‘theories in use’ and the attributes of software developers were identified at the level of the individual); this allowed the different themes, categories, and patterns to be identified in the case data (Yin, 2003). The case data was coded using a scheme based on a combination of open-coding (which permitted the identification of new themes and categories in the data) and the seed-categories suggested by a-priori concepts discussed above (cf. Stake, 1995). These are described as preformed codes, as per Seaman (1999), but, in agreement with Attride-Sterling (2001), it was acknowledged that these codes would change as themes emerged from the case.

FIELD OBSERVATIONS

The case chosen for study was a large US multinational telecommunications company, which designs and manufactures a range of products, from cell-phone switches and base stations, to mobile telephone products. Its software division develops software used in the monitoring, control and operation of mobile phone networks throughout the US, Europe, the Middle East, and Africa. It is significant that the organization’s policies and culture recently promoted software process improvement initiatives, such as Agile approaches. The unit of study selected in the case study (Yin, 2003) was a software development team of six software developers and a project manager. The team developed system administration applications for the installation, upgrade, and maintenance of a large application which monitored mobile phone networks. The team was purposively chosen for inclusion in the study, as it had decided to introduce an Agile software development methodology to help improve its software development processes and products. This team were early adopters of the Agile approach in this organization, as no other team in their section had attempted to adopt and use Agile methods.

The team made a collective decision to adopt a new Agile software development methodology, which was a hybrid of proven Agile methods, at the start of a new software development project. There was unanimous agreement among the team that this new methodology was required. The team members had reflected on their existing processes and methods and decided that an Agile approach would better meet the needs of the team and the project. Each software developer stated that they were looking forward to using the Agile method, one went as far as saying that he was “looking forward to ending the tyranny of the old approach.” The old approach, referred to by the

software developer, was the traditional waterfall method, with a heavy emphasis on documentation and predictability through plans. The Agile approach moves away from this emphasis and the software developers felt that they could more effectively do their job using the Agile method. In interviews, three software developers stated that this (the Agile method) was going to let them do their job properly; the other three software developers described the new approach as freedom from rigid and ineffective processes.

Initially, Agile was adopted successfully with iterations and user stories (a variant of requirements) being adopted by the team. While traditionally the team was expected to produce and stick with rigid plans, the use of iterations allows for more flexible planning which takes into account the reality of changing requirements. Documentation from Change Control Boards, over the previous five years, showed that the requirements could change by 30% up to 60%. Associated with that were the project plans from the previous five projects, which showed that the plan never reflected the reality of the project; this aligned with the continuous changes in requirements. It should be noted that the team were not trying to use the irrelevance of the plans as an excuse for missed deadlines or poor performance. Despite the fact that this team's requirements changed more than any other team, project management documents, and interviews with senior managers, showed this team as the most effective and productive in the organization. The move to user stories was a further example of the team's success with the initial adoption of Agile. User stories are simplified requirements documents; in fact, a user story can be as simple as a single line description along with associated development estimates, risks, etc. Traditionally, the team were required to produce volumes of requirements documents, most of which ended up "gathering dust", to quote one software developer, and "taking up space in my office", to quote another. The team, based on the Agile principle of eliminating unnecessary documentation, reviewed all their documentation requirements and drew a red pen through two-thirds of it. With support from the Quality Assurance department, they adopted new, more Agile, documents which only recorded what would actually be useful. The best example of the redundant documentation was the traditional requirement for pseudo code to be documented before coding, to guide the development. The reality was that the pseudo code was written after coding was complete, thus making it not just irrelevant but "a complete waste of my time." Pseudo code was the first part of requirements/design documents to be crossed out with the red pen.

The team initially discussed each aspect of the Agile adoption, used consent in decision making, and were committed to the adoption of Agile. During the first month of the project, each move to a new Agile process, and the new behaviors required, was formally discussed by the team in meetings. Recorded notes from these meetings showed that the team discussed the benefits from using a particular Agile process, while at the same time ensuring that the process could be used effectively by the team. Interviews with the individual software developers after the meetings showed that the software developers wanted to use the processes, and were quite strong in their support. For example, the use of iterations was discussed in the team. While the majority of Agile researchers recommend an iteration length of 4 weeks, the software developers discussed and decided upon a 6 week iteration. There is nothing in the existing Agile literature to suggest that 6 week iterations will be problematic (the opposite in fact) but what is noteworthy is that this was a collective decision after a team debate. While the six week period was not a universal choice – it was the majority choice though – two software developers who argued for 8 week iterations still

expressed support for the choice. It was specifically put to the two ‘dissenters’, by the researcher, that the team had gone against their choice. One replied that “at least with an Agile approach I get to express my view”, while the second stated the “you win some you lose some, at least with Agile I get to win some.” As such, they were demonstrating the Model II behaviors of open debate and commitment to decisions – effective learning. As the project progressed, though, it became clear that some aspects of the Agile adoption were not proceeding as planned, and the developers’ view of Agile changed from willingness to try, to one of rejection. Studying the software developers for the traits described above provided some illustration as to why problems were occurring, and why learning would be ineffective without the required new behaviors.

While ultimately the customers were multinational telecommunications companies, the immediate customer was in-house. The team developed system administration applications for the installation, upgrade, and maintenance of a large application which monitored mobile phone networks. The in-house customer used the team’s products internally to facilitate the other development teams. This created a unique situation where the team required input from other groups while at the same time providing functionality to these other teams to perform their development tasks – in effect the other teams were their customers.

Although the stereotypical view of software developers as ‘geeks’ was seen in the team, a low Social Need Strength (SNS) was not apparent. The software developers did have their own language and jargon which was unique to software developers. They had their own in-jokes which would have appeared odd to outsiders, as they were specific to software development and the individual members of the team. Despite this, there was no indication of a lack of social skills or a low SNS. While one of the software developers was introverted (‘odd’ to use the phrase of his fellow software developers), it would not be unusual to find one introvert in any group of six, no matter whether they were software developers or not. The remaining software developers maintained social relationships with their colleagues and members of other teams. They socialized together outside of work, either as a group or with spouses or partners, and could be considered as friends. Interviews with staff external to the group (quality group, senior project managers, and section managers) all described the team as a good collective unit – “possibly the best [software development] team in the company”. Another senior manager stated say how much he personally liked and respected the professionalism of individuals within the team. However, he qualified this statement by adding that they could be difficult to deal with as a group--they were “too tight” as a group, and were often suspicious of the motivations and attitudes of colleagues outside the group. This perspective was echoed by yet another senior manager, who described the group as “sometimes being overly defensive, but could be relied upon to deliver.”

The software developers never referred to themselves as artists, although some of their statements did show that they did not necessarily regard themselves as ‘stuffy engineers’. The software developers continually argued that their job was predominantly a creative one, where coding played a smaller part than most people assumed – “some managers in this place think that all we do is pump out code. It’s more about thinking than typing.” This was verified through observation. Approximately 30% of their time was spent writing code, the rest of their time being spent in discussion with fellow software developers, thinking, or what would be described as team bonding – be that simply going for coffee as a group or what could be described as water-

cooler sessions. These 'water-cooler sessions' were informal gatherings that took place, discussing everything from politics to sport, or personal issues.

Nine months of observations, allied with interviews with related social actors external to the team, did not highlight any specific concerns with an 'artistic temperament' in the individual software developers. At times, though, individual software developers were annoyed by feedback from individuals outside of the team. Collectively, the software developers did not appreciate feedback (negative or constructive) from those outside of their team. They did feel a personal ownership of their work and resented when others found fault with it. A typical response to negative feedback was "what would he know about it, he doesn't have any idea what we do." Variations of this were frequently heard throughout the project. Interestingly, the individual (outside the group) who could best be described as their internal customer did not cause offence with his feedback. This individual used the team's output internally and was respected by the team. In fact, his feedback and opinion was actively sought by the software developers. The Agile principle of customer involvement was thus accepted by the team. This is not to say that all feedback from their customer was accepted, but it was obvious that feedback from the customer was much more welcome than feedback from others external to the group. When asked about this, the software developers stated that the internal customer at least knew what they did, as opposed to their frequent complaints about others' opinions – "unlike the other people in this place, Jim [the internal customer] knows what he is talking about."

The software developers' reactions to feedback from those outside the team did suggest the possibility of cognitive dissonance, and was further investigated. Rather than listening to outsiders' feedback, they were automatically dismissed – "did you hear the latest brilliant suggestion from PMO [the Project Management Office]. I love it when they pretend they know what we're doing. We should have our own suggestion box for their ideas that automatically shreds the pages." As indicated, cognitive dissonance is seen where a person's self image, or understanding, is challenged and is avoided by adjusting belief. Rather than accepting that their work or opinion may be incorrect, the software developers avoided this by dismissing the views of outsiders which would cause dissonance. The possibility exists that the software developers were correct and that PMO suggestions should be ignored, but the PMO had successfully guided multiple releases companywide and were well regarded by the parent organization in the United States; it can therefore be assumed that not all of their ideas were bad. The software developers' negative view of 'outsiders' was not restricted to just the PMO group, most other groups suggestions were dismissed as an annoyance – "another stupid idea from the GUI group. Why don't they stick to messing up their own stuff?"

Further investigation of dissonance in this study concentrated on self-testing, which was identified by Weinberg as posing particular problems for software developers (Weinberg, 1971). Following the Agile principle of testing, tests were developed in collaboration with the customer (the internal customer in this case) and tests were run by the software developers themselves. Finally, the customer tested the finished product, or release. Discussions with the internal customer (and other members of his group) portrayed the Agile team as effective in testing – "never has one of their PR's [bugs] stopped a release. You couldn't say that for the other teams." Agile's reliance on self-testing did not appear to have a detrimental effect on the final product.

Analysis of test reports and statistics found no major differentiation between the Agile team and other non-Agile teams; in fact the Agile team was rated highly with regard to reliable code. From this, it can be deduced that self-testing, and the possibility of cognitive dissonance affecting testing, was not having any noticeable impact on testing by the Agile team.

The one noticeable impact of software developers' traits on the introduction of Agile, and the learning activity associated, was in pair-programming; in pair programming the software developers code in pairs, one typing and one reviewing the work. After requirements had been defined and documented, the concept of pair-programming was introduced to the software developers. It was met with universal resistance to the extent that it was not used by the team. Although other aspects of Agile had been adopted by the team, pair programming was never implemented and the reasoning behind this was further investigated. Each software developer was asked why pair programming was resisted and a common thread was visible in their responses. Each of the other software developers explained that they had too little time for two software developers to do one task – “waste of my time”, “too much external pressure for speed”. When it was pointed out that existing research had shown how pair programming could be more productive, this was collectively dismissed as outsiders not understanding how this team works – “what would he [referring to author of a book on XP] know about how we work here.” This dismissal, of the work of ‘outsiders’, highlighted the possibility of cognitive dissonance. The individual software developers were confronted by the researcher with observations of other episodes of activities which could be regarded as ineffective use of time – the justification they used to dismiss pair-programming. Each finally admitted that their concern with pair programming was actually the possibility of their pair seeing errors or inadequacies in their work. This was not a lack of trust in their fellow developers, the software developers explicitly stated that they trusted their colleagues and this was demonstrated by the openness of code development where all software developers were allowed to make changes to their colleagues' code. This was rather a problem with the software developers striving to ensure a positive impression of their developing abilities from their colleagues. Rather than risking their colleagues finding fault in their work, they argued against pair programming based on a false claim that it would waste their time – the software developers had convinced themselves that this was true rather than accepting their own faults or insecurities.

From a learning perspective, both Model I (as typified as single-loop learning) and Model II (as typified as double-loop learning) were observed (see Table 3 below). Contrary to the stereotype, the software developers collectively constituted an effective and cohesive team, demonstrating an ability to discuss issues openly within the group. They were also committed to decisions taken – both of which are Model II behaviors. Conversely, individual software developers suppressed negative feedback (through cognitive dissonance) and appeared to value winning – also described as avoiding being seen as incompetent. These behaviors are associated with Model I behavior and are an impediment to learning – in this case the adoption of an Agile methodology.

Model I Behavior	Model II Behavior
Suppression of negative feeling. Limited feedback. Censorship. The software developers rejected and censored the opinions of “outsiders”	Valid information The software developers were willing to listen to input from the customer
Importance of rationality, stability, and the avoidance of risk. Rationality is used to justify one’s own views and discount those of others. The move to pair programming came with the risk for the individual software developers that colleagues would find fault with their work. The risk was avoided and cognitive dissonance created a rational for this avoidance.	Free and informed choice / Open debate The decision to adopt Agile was informed and a collective decision. The absence of a low SNS meant that software developers were willing to get involved in group discussions and debate
Winning (not losing or being seen as incompetent) The fear of being seen as incompetent by their colleagues negated any benefits of pair programming	Commitment to decisions and actions There was initial commitment to adopting Agile, yet there was a lack of commitment to pair programming due to the inherent characteristics of the software developers

Table 3: Learning and lack of learning from the software developers

CONCLUSIONS

While the overall project was a success, the adoption itself was not completely successful, as described above. Although this may appear paradoxical, the project was a success because of a talented team which succeeded without the full use of Agile. It is debatable if a less skilled team could have had an unsuccessful adoption while still achieving project success. This research, though, examines the success of the adoption. By viewing learning as behavioral change (as per Argyris), it can be seen that the learning required in the adoption of Agile was not completely successful in that the new Model II behaviors were not fully acquired.

Software developers may differ from other professionals, but the stereotypical descriptions of them were not applicable in this research. Despite previous studies suggesting that software developers have a low SNS, and this research originally suggesting that this may affect the learning exercise of the adoption of Agile, the team of software developers studied in this case

study did not demonstrate a low SNS. It should be noted that most of the research on software developers' SNS occurred decades ago, prior to the use of Agile, so a new generation of software developers do not appear to have the same problem in social or team based situations - a prerequisite for Agile projects. It cannot be determined from this case if a low SNS would have the negative impacts predicted, yet it is noteworthy that the low SNS did not occur as would have been assumed; in fact the developers demonstrated a high SNS. Further, the description of teams of software developers as an 'artists' colony' was not disproved in this research; the software developers did show traits which would align with this view, but these trait did not appear as a major concern in this particular adoption of an Agile methodology. Further research would be required in other teams to find if the artistic temperament dominates and, if so, what impact it has. The research model, in Figure 1, predicted a low SNS and an artistic perspective as having a negative impact. They were not observed to be present or impacting on the adoption, so it cannot be determined from this case if and what their impact would be. It is worth noting, though, that they may not impact at all.

Cognitive dissonance was viewed as a problem for software developers in previous research by Weinberg (1971), but this concerned software developers testing their own code. Again, the findings did not show this to be of concern when adopting an Agile methodology. Cognitive dissonance, however, was evident, but not where initially expected. It was created in the individual software developers, specifically when the software developers received feedback or suggestions from those external to the team. This is the sole example of an artistic trait impacting on the Agile project; it was predicted that the artistic attributes of developers would cause problems with others evaluating their work, and this was seen. 'Outsiders' views were dismissed as irrelevant, rather than the software developers accepting the possibility that they may need to change their views or behavior. This is cognitive dissonance, where the real problems are hidden or excused. Significantly, however, feedback from the customer was accepted by the software developers, which was positive in terms of Agile's adoption. The research model, though, predicted a negative impact from cognitive dissonance in the area of testing. This was not seen in the case.

Although feedback from those external to the team was problematic for an Agile adoption, the primary concern seen was with internal feedback or evaluation. The software developers feared negative feedback from their fellow team members; this was something seen in previous research (Sawyer & Guinan, 1998) (albeit these studies were not Agile specific). Interestingly, while the developers feared negative feedback from their fellow developers, they were open to feedback from the customer. While the customer was respected by the team, he was not regarded as a peer, or an actual member of the team. As such, his feedback was not internal feedback, so did not cause the same problems. The customer was seen on the fine line that separated outsiders from peers; he was neither a peer nor an outsider. Although a successful balance between respect for the customer and the necessity for distance between the team and the customer was achieved in this case study, it needs to be acknowledged that this could be problematic in other Agile projects. If the customer is too removed from the team then the necessary respect for the customer and their opinions may not exist. Conversely, if the customer is too integrated in the team, then the developers may adopt the same avoidance of negative feedback from a peer. Re-examining Figure 1, peer motivation was predicted as having a positive impact on the adoption/learning. In

fact, this case shows how peer motivation, specifically the desire for recognition from peers, actually had a negative impact on the adoption.

In this case study, in an effort to avoid negative internal feedback (from their peers), the software developers displayed cognitive dissonance by deliberately blaming time constraints on the failure to adopt pair-programming. Dissonance arises where a person's self image is challenged and is avoided by adjusting beliefs. As such, the possibility of fellow software developers seeing faults in their work (the likelihood of which increased while working closely together in pair programming) created a challenge to the software developers' self-image. The software developers reduced this dissonance by dismissing pair programming as ineffective use of resources; without true reflection on the reasons, though, learning could not take place. Interestingly, the cohesion of the group, demonstrated by the collective purported view that pair programming was a waste of resources, helped the software developers to reduce dissonance. It is easier to decrease dissonance when you appear to be consistent with your peers, especially in a cohesive unit. The impact of cohesion on an individual's dissonance reduction was noted by psychologists in the early 1960's (Brehm & Cohen, 1962). It is relevant to this study as it further shows the software developers as a cohesive unit as opposed to software developers having a low SNS. As such, returning to the research question, the major trait that negatively impacted on the behavioral changes required for the learning inherent in an Agile adoption was the fear of negative feedback from peers, and cognitive dissonance arose in an attempt to disguise this trait. The research model in Figure 1 predicted cognitive dissonance impacting negatively on the requirement for self-testing of code. Again, this was not seen, but as described above, cognitive dissonance actually added to the negative impact of peer motivation, by disguising the real problem.

It is not the aim of this study to make generalizations from this single case. The nature of the present research necessitated an approach that was descriptively rich and in-depth, rather than one which could make generalizable statements. Based on this case study, we argue that the output highlights important concerns for those considering the adoption of Agile. There are no certainties that similar issues will arise in other adoptions, but the issues described here do merit consideration by stakeholders striving to ensure effecting learning in an adoption.

RECOMMENDATIONS

From a learning perspective, while the software developers' traits exhibited both a positive and negative impact on the adoption of Agile, it is clear that much of the negative impact could be mitigated by the closer involvement and questioning of a project manager or team leader. If the software developers' views and traits had been questioned, there is a greater chance that undesirable behavior could have been addressed, and learning could have occurred. However, software developers themselves could not do this – cognitive dissonance is not easily corrected by the individual concerned. As previously noted, Agile projects require less involvement and instruction from a project manager than in traditional projects. It is likely that this reduction in project manager involvement, aligned with the software developers' requirements for peer recognition, ultimately led to the failure to introduce pair programming. This outcome in the adoption of Agile must be balanced with the other positive software developer traits that aligned well with the Agile philosophy. With more involvement from the project manager, Model I type

learning behaviors could be identified by the team for what they are, and a transition to effective learning through Model II behaviors made, thereby increasing the prospects of success for the Agile adoption. This necessitates a manager skilled in reflection, and in enabling reflection in others. In essence, this ties back to Argyris' original description of his theory of learning, where he recommends intervention to help individuals recognize ineffective learning.

In addition, while Agile project managers must use a new approach to management, moving away from 'engineering' based approaches, there needs to be a balance between empowerment (delegation of control to software developers) and the necessity for intervention to ensure reflection by the developers. To ensure reflection by the individual software developers, the project manager must accept that the stereotypical low SNS of software developers may no longer be accurate, and acknowledge the impact that the team can have on the individual's ability to adapt new behaviors.

REFERENCES

1. Acuna, S., & Juisto, N. (2004). Assigning people to roles in software projects *Software: Practice and experience*, 34(7), 675-696.
2. Argyris, C. (1976). Single-loop and double-loop models in research on decision making. *Administrative Science Quarterly*, 21(3), 363-375.
3. Argyris, C. (1994). Good communication that blocks learning. *Harvard Business Review*, 72(4), 77-85.
4. Argyris, C. (1995). Action science and organizational learning. *Journal of Managerial Psychology*, 10(6), 20-26.
5. Argyris, C. (1997). Double loop learning in organizations. *Harvard Business Review*, 55(5), 115-125.
6. Argyris, C. (2002). Double loop learning, teaching, and research. *Academy of Management Learning and Education*, 1(2), 206-218.
7. Argyris, C., & Schon, D. (1978). *Organizational learning: A theory of action perspective*. MA, USA: Addison-Wesley.
8. Astels, D., Miller, G. and Movak, M. (2002) *A practical guide to extreme programming*. NJ, USA: Prentice Hall.
9. Attride-Sterling, J. (2001). Thematic networks: an analytical tool for qualitative research. *Qualitative Research*, 1(3), 385-405.
10. Augustine, S. (2005). *Managing Agile Projects*. IA, USA: Prentice Hall.
11. Back, K., & Seaker, R. (2004). Project performance: Implications of personality preferences and double loop learning. *Journal of American Academy of Business*, 4(1), 292-297.
12. Badawy, M. (1997). The role of the technical manager. In R. Katz (Ed.), *The human side of managing technological innovation* (229-236). NY, USA: Oxford University Press.
13. Beck, K. (2000). *Extreme programming explained. Embrace change*. NY, USA: Addison Wesley.
14. Blackman, D., Connelly, J., & Henderson, S. (2004). Does double loop learning create reliable knowledge? *The Learning Organization*, 11(1), 11-27.
15. Bødker, K., & Pedersen, J. (1991). Workplace cultures: Looking at artifacts, symbols and practices. In K. Greenbaum & M. Kyng (Eds.), *Design at Work: Cooperative Design of Computer Systems* (121-136). NJ, USA: Lawrence Erlbaum Associates.
16. Boehm, B., & Turner, R. (2004). *Balancing agility and discipline*. Pearson Education, MA, USA.
17. Bokeno, R. (2003). The work of Chris Argyris as critical organization practice. *Journal of Organizational Change Management*, 16(6), 633-649.
18. Boody, D., & Buchanan, D. (1992). *Take the lead. Interpersonal skills for project managers*. Hertfordshire, UK: Prentice Hall.

19. Bossavit, L. (2002). The unbearable lightness of programming: A tale of two cultures. *Cutter IT Journal*, 15(9), 5-11.
20. Brehm, J., & Cohen, A. (1962). *Explorations in cognitive dissonance*. NY, USA: John Wiley & Sons.
21. Capretz, L. (2002). Are software engineers really engineers. *World Transactions on Engineering and Technology Education*, 1(2), 233-235.
22. Carmel, E., & Sawyer, S. (1998). Packaged software development teams: What makes them different. *Information Technology and People*, 11(1), 7-19.
23. Coffee, P. (2001). No more excuses: Innovation technology and irrelevant tangents. In L. Constantine (Ed.), *Beyond chaos: The expert edge in managing software development*. NJ, USA: Addison-Wesley.
24. Cooper, A. (2004). *The inmates are running the asylum*. IN, USA: Sams Publishing.
25. Coram, M., & Bohner, S. (2005). *The impact of agile methods on software project management*. Paper presented at the IEEE International Conference on the Engineering of Computer-Based Systems, Greenbelt, MD, USA.
26. Couger, J., & McIntyre, S. (1988). Motivation norms of knowledge engineers compared to those of software engineers. *Journal of Management Information Systems*, 4(3), 82-93.
27. Couger, J., Zawacki, R., & Oppermann, E. (1979). Motivation levels of MIS managers versus those of their employees. *MIS Quarterly*, 3(3), 47-56.
28. Curtis, B., Krasner, H., & Iscoe, N. (1988). A field study of the software design process for large systems. *Communications of the ACM*, 31(11), 1268-1287.
29. Denhardt, R., Denhardt, J., & Aristigeuta, M. (2002). *Managing behavior in public and nonprofit organizations*. CA, USA: Sage Publications.
30. Drobka, J., Nofz, D., & Raghu, R. (2004). Piloting XP on four mission-critical projects. *IEEE Software*, 21(6), 70-75.
31. Easterby-Smith, M., & Lyles, M. (2003). Re-reading organizational learning: Selective memory, forgetting, and adaptation. *Academy of Management Executive*, 17(2), 51-55.
32. Ezey, P. (2003). Integration and its challenges in participant observation. *Qualitative Research*, 3(2), 191-205.
33. Fitzgerald, B. (1997). The use of systems development methodologies in practice: a field study. *Information Systems Journal*, 7(3), 210-212.
34. Furumo, K., & Melcher, A. (2006). The importance of social structure in implementing ERP systems: A case study using Adaptive Structuration theory. *Journal of Information Technology Case and Application Research*, 8(2), 39-58.
35. Goleman, D. (1996). *Vital Lies Simple Truths: The Psychology of Self-Deception*. NY, USA: Simon and Schuster Books.
36. Gordon, S. (2008). The case for case-based research. *Journal of Information Technology Case and Application Research*, 10(1), 1-6.
37. Greenwald, A., & Ronis, D. (1978). Twenty years of cognitive dissonance: Case study of the evolution of a theory. *Psychological Review*, 85(1), 53-57.
38. Grossman, F., Bergin, J., Leip, D., Merrit, S., & Gotel, O. (2004). *One XP experience: introducing agile (XP) software development into a culture that is willing but not ready*. Paper presented at the Conference of the Centre for Advanced Studies on Collaborative research Ontario, Canada.
39. Hazzan, O., & Tomayko, J. (2003). *The reflective practitioner perspective in extreme programming*. Paper presented at the Extreme programming and agile methods – Xp/Agile universe 2003, New Orleans.
40. Highsmith, J. (2004). *Agile project management*. MA, USA: Pearson Education.
41. Hodgetts, P. (2004). *Refactoring the development process: Experiences with the incremental adoption of Agile practices*. Paper presented at the Agile Development Conference (ADC04) Salt Lake City.
42. Howard, A. (2001). On site: Software engineering project management. *Communications of the ACM* 44(5), 23-24.
43. Humphrey, W. (1996). Making software manageable. *Crosstalk*, 9(12), 3-6.

44. Hunt, A., & Thomas, D. (2000). *The pragmatic programmer*. MA, USA: Addison Wesley Longman.
45. Hunt, A., & Thomas, D. (2002). Software archaeology. *IEEE Software*, 19(2), 20-22.
46. Jones, M., & McLean, E. (1970). Management problems in large-scale software development projects. *Industrial Management Review*, 11(3), 1-16.
47. Jorgensen, D. (1989). *Participant observation: A methodology for human studies*. CA, USA: Sage Publications.
48. Joshi, K. (2005). Understanding user resistance and acceptance during the Implementation of an order management system: A case study using the equity implementation model. *Journal of Information Technology Case and Application Research*, 7(1), 6-20.
49. Kahkonen, T., & Abrahamsson, P. (2003). *Digging into the fundamentals of extreme programming building the theoretical base for agile methods*. Paper presented at the 29th Euromicro Conference, Antalya, Turkey.
50. Kitchenham, B., Pickard, L., & Pfleeger, S. (1995) Case studies for method and tool evaluation. *IEEE Software*, 12(4), 52-62.
51. Klein, K., & Myers, M. (1999). A Set of Principles for Conducting and Evaluating Interpretive Field Studies in Information Systems. *MIS Quarterly*, 23(1), 67-94.
52. Koutsoukos, G. (2006). *Mentality patterns: Capturing and dealing explicitly with recurring turns of mind in software development*. Paper presented at the 13th European Conference EuroSPI, Joensuu, Finland.
53. Larsen, T. (2001). The phenomenon of diffusion. In M. Ardis & B. Marcolin (Eds.), *Diffusing software product and process innovations* (35-50). MA, USA: Kluwer Academic Publishers.
54. Layman, L., Williams, L., & Cunningham, L. (2004). *Exploring extreme programming in context: an industrial case study*. Paper presented at the Agile Development Conference (ADC04), Salt Lake City.
55. Layman, L., Williams, L., & Cunningham, L. (2006). Motivations and measurements in an agile case study. *Journal of Systems Architecture*, 52(11), 654-667.
56. Licker, P. (1983). The Japanese Approach: A Better Way To Manage Programmers. *Communications of the ACM*, 26(9), 631-636.
57. Licker, P. (1985). *The Art of Managing Software Development People*. NY, USA: John Wiley & Sons.
58. Lindstrom, L., & Jeffries, R. (2004). Extreme Programming and Agile software development methodologies. *Information Systems Management*, 21(3), 41-52.
59. Maurer, F., & Martel, S. (2002). Extreme Programming: Rapid Development for Web-Based Applications. *IEEE Internet Computing*, 6(1), 86-90.
60. McAvoy, J., & Butler, T. (2007) *A failure to learn in a software development team: The unsuccessful introduction of an Agile method* Paper presented at the 16th International Conference on Information Systems Development Galway, Ireland.
61. Molla, A., & Bhalla, A. (2006). Business transformation through ERP: A case study of an Asian company. *Journal of Information Technology Case and Application Research*, 8(1), 34-54.
62. Natsu, H., Favela, J., Morán, A., Decouchant, D., & Martinez-Enriquez, A. (2003). *Distributed Pair Programming on the Web* Paper presented at the 4th Mexican International Conference on Computer Science Tlaxcala, Mexico.
63. Olson, D., & Stimmel, C. (2002). *The manager pool. Patterns for radical leadership*: Addison-Wesley. NJ, USA.
64. Poole, C. (2004) *Distributed product development using extreme programming* Paper presented at the Extreme programming and agile processes in software engineering 5th International Conference Garmisch-Partenkirchen, Germany.
65. Raymond, E. (1999). *The cathedral and the bazaar*. CA, USA: O'Reilly & Associates.
66. Riehle, D. (2001). A comparison of the value systems of Adaptive Software Development and Extreme Programming: How methodologies may learn from each other. In G. Succi & M. Marchesi (Eds.), *Extreme Programming explained* (35-50). Boston, USA: Addison-Wesley.

67. Robinson, H., & Sharp, H. (2005). *The social side of technical practices*. Paper presented at the 6th International Conference on Extreme Programming and Agile processes in software engineering (XP2005), Sheffield, UK.
68. Rumpe, B., & Schroeder, A. (2002). *Quantitative survey on Extreme Programming projects*. Paper presented at the International Conference on Extreme Programming and Flexible Processes in Software Engineering (XP2002), Alghero, Italy.
69. Salaway, G. (1987). An organizational learning approach to information systems development. *MIS Quarterly*, 11(2), 245-264.
70. Sawyer, S., & Guinan, P. (1998). Software development: Processes and performance. *IBM Systems Journal*, 37(4), 552-569.
71. Schuh, P. (2001). Recovery, Redemption, and Extreme Programming. *IEEE Software*, 18(6), 34-41.
72. Schuh, P. (2004) *Integrating agile development in the real world*. NY, USA: Delmar Thomson Learning
73. Schwaber, K. (2002). When and where Agile succeeds. *Cutter IT Journal*, 15(9), 22-27.
74. Seaman, C. (1999). Qualitative methods in emperical studies of software engineering. *IEEE Transactions on Software Engineering*, 25(4), 557-572.
75. Simons, M., & Nadkarny, C. (2002). Extreme India. *Cutter IT Journal*, 15(9), 28-33.
76. Spayd, M. (2003). *Evolving agile in the enterprise: implementing XP on a grand scale*. Paper presented at the Agile Development Conference, Salt Lake City, Utah.
77. Stake, R. (1995). *The art of case study research*. CA, USA: Sage Publications.
78. Stephens, M., & Rosenberg, D. (2003). *Extreme Programming refactored: The case against XP*. Heidleberg, Germany: Springer-Verlag.
79. Turk, D., France, R., & Rumpe, B. (2005). Assumptions underlying Agile software development processes. *Journal of Database Management*, 16(4), 62-87.
80. Von Bergen, C., & Kirk, R. (1978) Groupthink: When too many heads spoil the decision, *Management Review*, 67(3), 44-49.
81. Vroom, V. (1974) A normative model of managerial decision-making, In Pugh, D (Ed.), *Organization theory. Third Edition* (309-328). London, UK: Penguin Books.
82. Walsham, G. (1995). Interpretive Case Studies in IS Research: Nature and Method. *European Journal of Information Systems*, 4(1), 74-81.
83. Weinberg, G. (1971). *The psychology of computer programming*. NY, USA: Van Nostrand Reinhold Co.
84. Wendorff, P. (2002). *Organisational Culture in Agile Software Development*. Paper presented at the Product Focused Software Process Improvement : 14th International Conference, PROFES 2002, Rovaniemi, Finland.
85. Wieggers, K. (2001). First things first: A project managers primer. In L. Constantine (Ed.), *Beyond chaos: The expert edge in managing software development* (49-55). NJ, USA: Addison-Wesley.
86. Workman, J. (1997). Engineers' interactions with marketing groups in an engineering-driven organization. In R. Katz (Ed.), *The human side of managing technological innovation* (535-549). NY, USA: Oxford University Press.
87. Yin, R. (2003). *Case Study Research: Design and Method*. CA, USA: Sage Publications.
88. Yourdon, E. (1993). *Decline and fall of the American programmer*. NJ, USA: Prentice Hall Inc.

John McAvoy is a lecturer in Business Information Systems at University College Cork, Ireland. Prior to lecturing, John had a variety of roles in the Information Systems field, ranging from systems administration to managing software development teams.

Tom Butler is a Senior Lecturer in Business Information Systems at University College Cork, Ireland. Before lecturing, Tom had an extensive career in the telecommunications industry. Tom has also successfully managed a development project for the United Nations.